Navigating the Future of Online Shopping

Made By - Devang Magare

Project Overview

This project analyzes an e-commerce dataset to understand sales performance and customer behavior. SQL is used to extract and compute insights from structured data, while Python is used to visualize the same results for better interpretation. Using both tools together helps validate the analysis and ensures consistency in results across different approaches.

Deciphering the Digital Marketplace: Dataset Overview

Customers Table

Comprehensive details of individual customers, including geographical locations, providing a foundational layer for demographic analysis.

Orders Table

Records of order dates and their current statuses, crucial for tracking purchasing timelines and operational efficiency.

Order Items Table

Detailed breakdown of product-level sales and pricing, offering granular insight into individual transaction values.

Products Table

Categorization of various products, enabling analysis of popular segments and inventory management.

Payments Table

Information on payment values and methods, essential for understanding revenue streams and financial flows.

This dataset offers a realistic snapshot of online shopping activities, allowing for in-depth analysis of consumer trends and business performance.

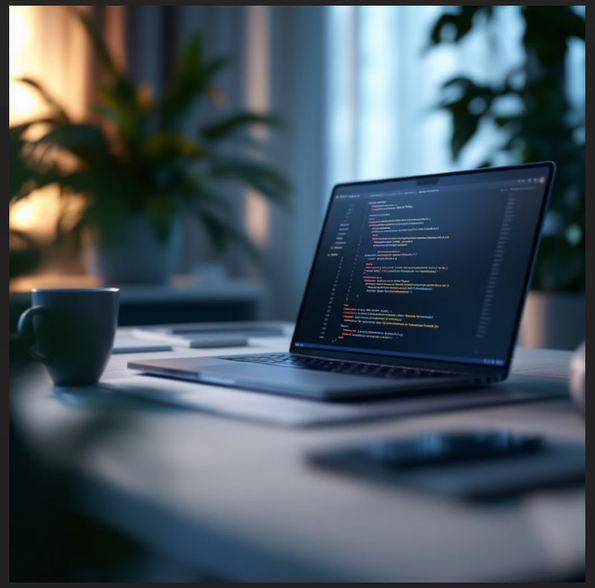
Analytical Arsenal: Tools & Methodology

SQL (MySQL Workbench)

- Efficient data extraction using advanced join operations.
- Powerful aggregations and grouping for summarized insights.
- Complex queries for in-depth retention analysis.

Python (Pandas & Matplotlib)

- Robust data manipulation capabilities with Pandas.
- Dynamic visualization of trends using Matplotlib.
- Critical cross-validation of SQL-derived results for accuracy.



SQL + PYTHON

BASIC PROBLEM

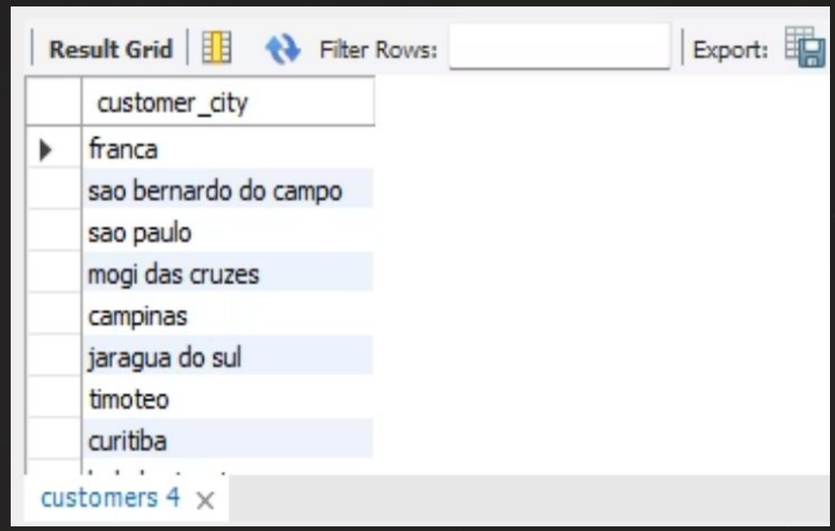
Q1 .List all unique cities where customers are located

SQL Query

```
select distinct customer_city
from customers ;
```

Python Query

```
unique_cities = customers['customer_city'].unique()

unique_cities = customers[['customer_city']].drop_duplicates()

unique_cities.index = range(1, len(unique_cities) + 1)

unique_cities
```



	customer_city
1	franca
2	sao bernardo do campo
3	sao paulo
4	mogi das cruzes
5	campinas
...	...
4115	siriji
4116	natividade da serra

SQL + PYTHON

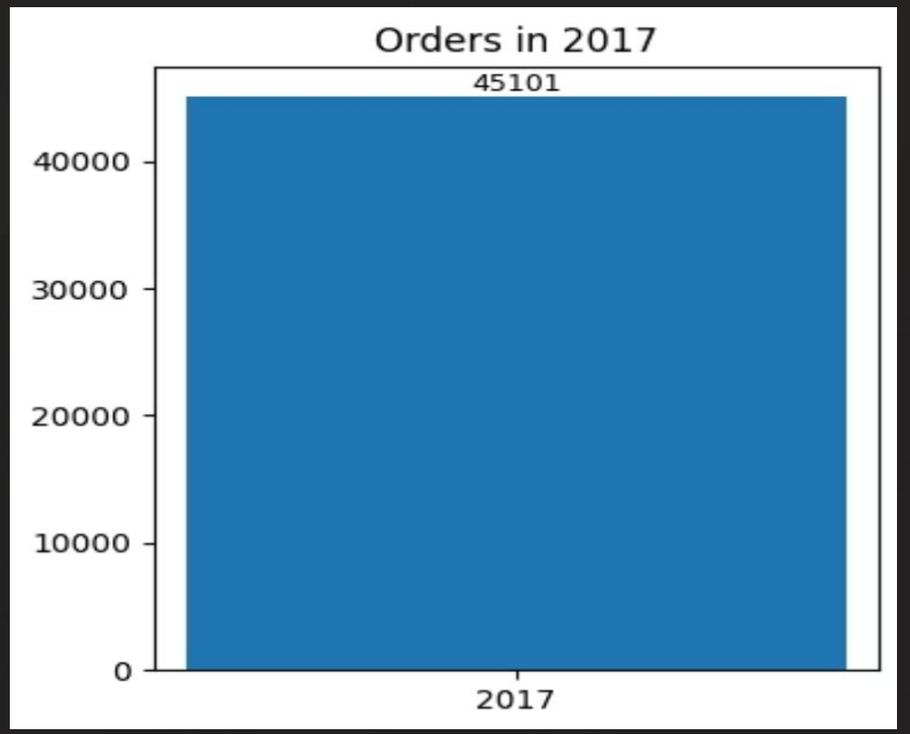
Q2. Count the number of orders placed in 2017

SQL Query

```
SELECT COUNT(*) AS total_orders_2017 FROM orders

 WHERE YEAR(order_purchase_timestamp) = 2017;
```

Python Query

```
plt.figure(figsize=(4,4))

bars = plt.bar(['2017'], [count_2017])

plt.title('Orders in 2017')

plt.bar_label(bars, labels=[count_2017], fontsize=9)

plt.tight_layout()
plt.show()
```



SQL + PYTHON

Q3. Find the total sales per category

SQL Query

```
SELECT products.product_category, SUM(order_items.price)
 AS total_sales FROM order_items JOIN products ON

 order_items.product_id = products.product_id GROUP BY
products.product_category;
```

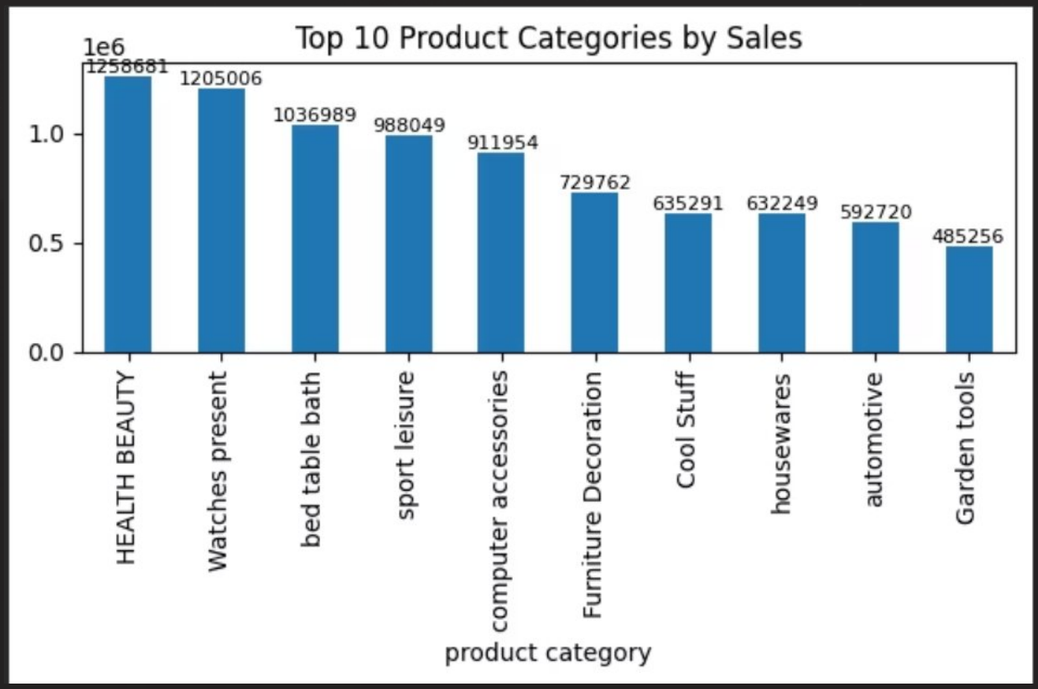
Python Query

```
plt.figure(figsize=(6,4))

bars = sales_cat.plot(kind='bar')

plt.title('Top 10 Product Categories by Sales')

plt.bar_label(bars.containers[0], fmt='%.0f', fontsize=8)
plt.tight_layout()
plt.show()
```

Result Grid | Filter Rows:

product_category	total_sales
HEALTH BEAUTY	1258681.3399999724
sport leisure	988048.9700000428
Cool Stuff	635290.8500000002
computer accessories	911954.3200000391
Watches present	1205005.6799999962
housewares	632248.6600000234
electronics	160246.739999998
toys	483946.60000000196

Result 3

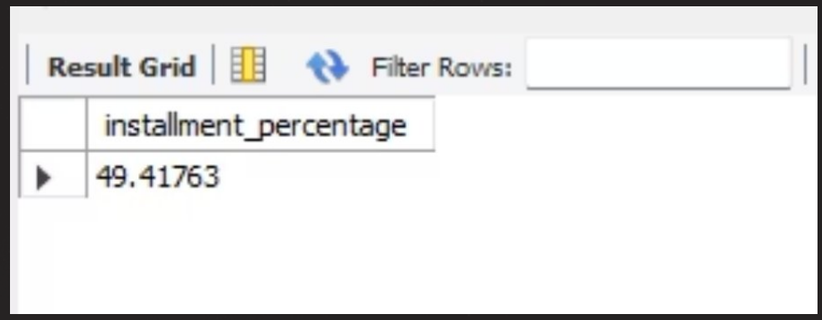


SQL + PYTHON

Q4. Calculate the percentage of orders that were paid in installments.

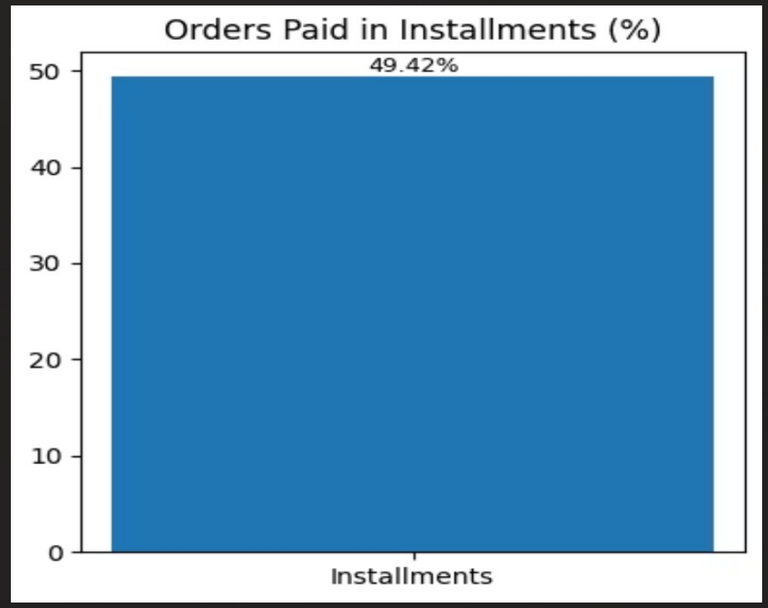
SQL Query

```
SELECT (COUNT(CASE WHEN payment_installments > 1

 THEN 1 END) * 100.0 / COUNT(*)) AS

 installment_percentage FROM payments;
```



Python Query

```
plt.figure(figsize=(4,4))

bars = plt.bar(['Installments'], [installment_pct])

plt.title('Orders Paid in Installments (%)')

plt.bar_label(bars, fmt='%.2f%%', fontsize=9)
plt.tight_layout()
plt.show()
```



SQL + PYTHON

Q5. Count the number of customers from each state.

SQL Query

```
SELECT customer_state, COUNT(*) AS total_customers

 FROM customers GROUP BY customer_state ORDER BY
total_customers DESC;
```

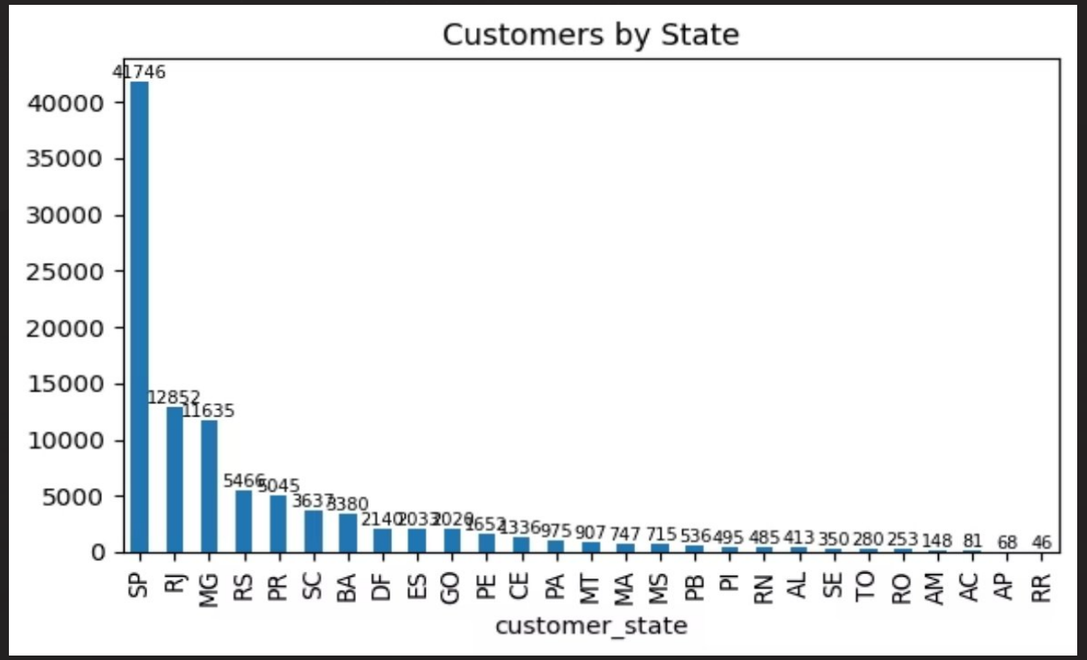
Result Grid | Filter Rows:

customer_state	total_customers
SP	41746
RJ	12852
MG	11635
RS	5466
PR	5045
SC	3637
BA	3380
DF	2140

Result 5

Python Query

```
plt.figure(figsize=(6,4))

bars = state_counts.plot(kind='bar')

plt.title('Customers by State')

plt.bar_label(bars.containers[0], fontsize=7)
plt.tight_layout()
plt.show()
```



SQL + PYTHON

INTERMEDIATE PROBLEM

Q1. Calculate the number of orders per month in 2018.

SQL Query

```
SELECT MONTH(order_purchase_timestamp) AS
order_month, COUNT(order_id) AS total_orders FROM orders

 WHERE YEAR(order_purchase_timestamp) = 2018 GROUP
BY MONTH(order_purchase_timestamp) ORDER BY
order_month;
```

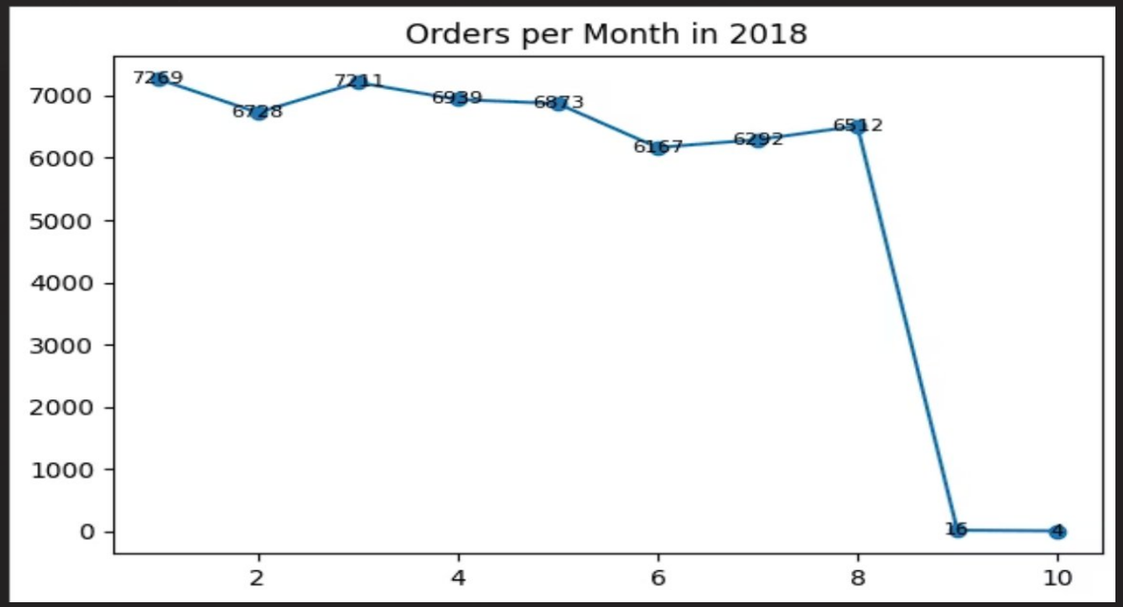
Result Grid | Filter Rows:

order_month	total_orders
1	7269
2	6728
3	7211
4	6939
5	6873
6	6167
7	6292
8	6512

Result 6

Python Query

```
plt.figure(figsize=(6,4))

plt.plot(monthly_orders.index, monthly_orders.values,
marker='o')

plt.title('Orders per Month in 2018')

for x, y in zip(monthly_orders.index,
monthly_orders.values):
    plt.text(x, y, y, ha='center', va='center', fontsize=8)

plt.tight_layout()

plt.show()
```



SQL + PYTHON

Q2. Find the average number of products per order, grouped by customer city.

SQL Query

```
SELECT c.customer_city, COUNT(oi.product_id) / COUNT(DISTINCT
o.order_id) AS avg_products_per_order FROM orders o JOIN

order_items oi ON o.order_id = oi.order_id JOIN customers c ON

o.customer_id = c.customer_id GROUP BY c.customer_city ORDER BY

avg_products_per_order DESC LIMIT 10;
```

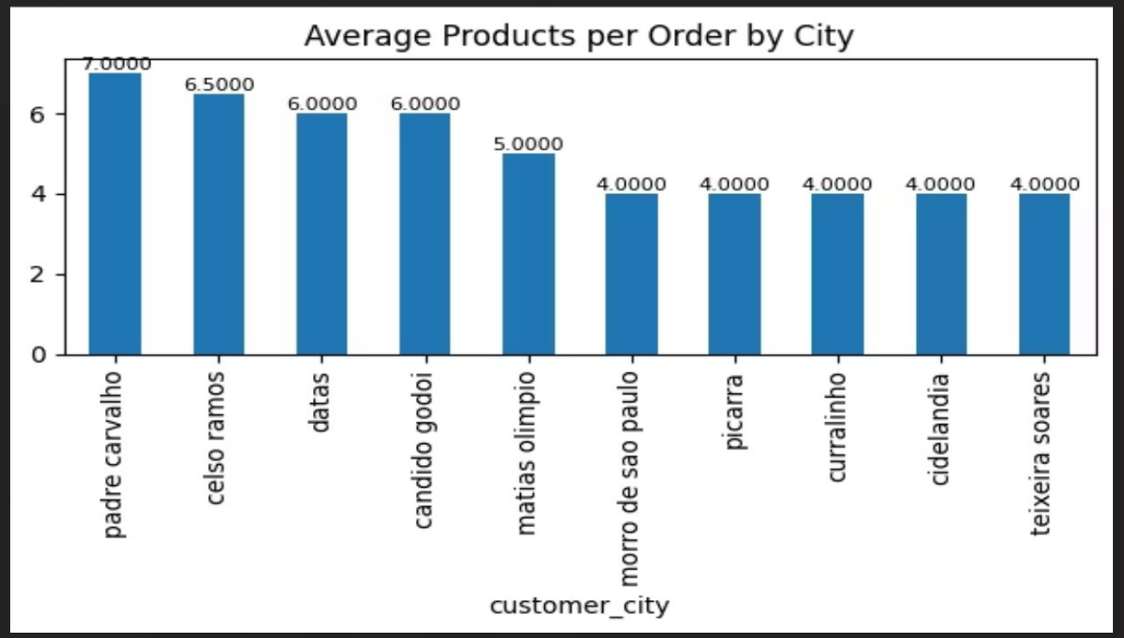
Result Grid | Filter Rows:

customer_city	avg_products_per_order
padre carvalho	7.0000
celso ramos	6.5000
datas	6.0000
candido godoi	6.0000
matias olimpio	5.0000
teixeira soares	4.0000
morro de sao paulo	4.0000
picarra	4.0000

Result 7

Python Query

```
plt.figure(figsize=(6,4))

bars = city_avg.head(10).plot(kind='bar')

plt.title('Average Products per Order by City')

plt.bar_label(bars.containers[0], fmt='%.4f', fontsize=8)

plt.tight_layout()
plt.show()
```



SQL + PYTHON

Q3. Calculate the percentage of total revenue contributed by each product category.

SQL Query

```
SELECT p.product_category, ROUND( SUM(oi.price) * 100.0 /
 (SELECT SUM(price) FROM order_items), 2 ) AS revenue_percentage

 FROM order_items oi JOIN products p ON oi.product_id =
p.product_id GROUP BY p.product_category

ORDER BYrevenue_percentage DESC;
```

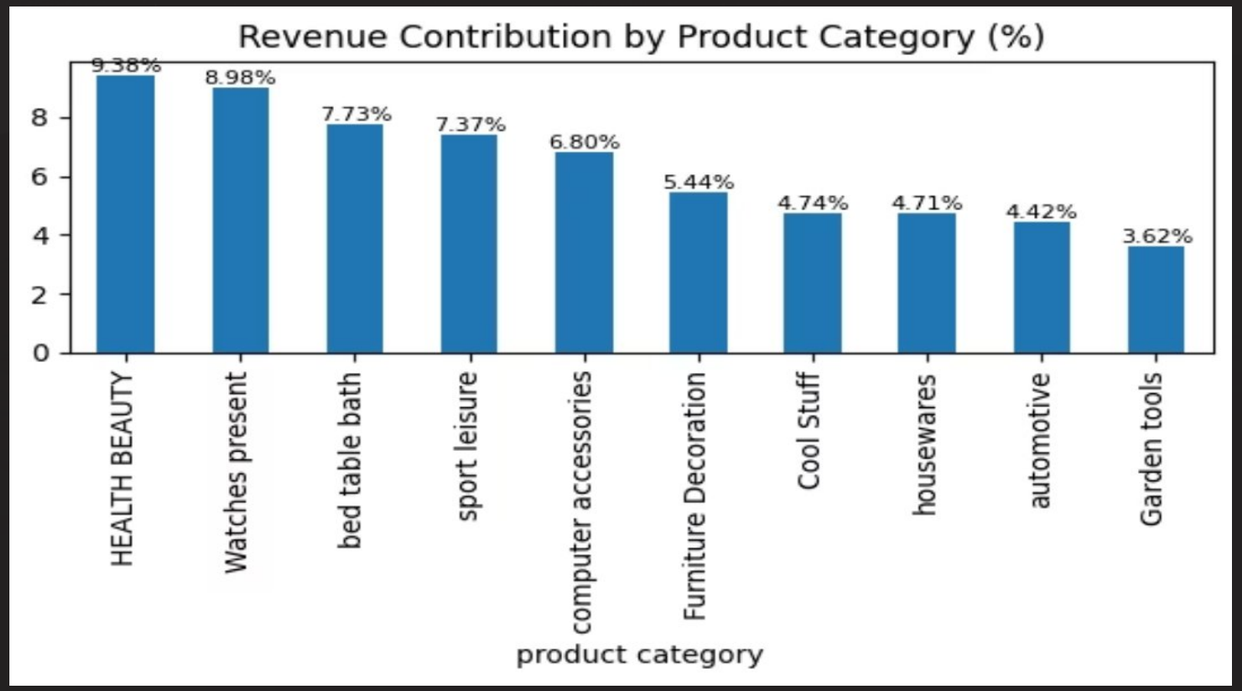
Result Grid | Filter Rows:

product_category	revenue_percentage
HEALTH BEAUTY	9.26
Watches present	8.87
bed table bath	7.63
sport leisure	7.27
computer accessories	6.71
Furniture Decoration	5.37
Cool Stuff	4.67
housewares	4.65

Result 8

Python Query

```
plt.figure(figsize=(6,4))

bars = percent_revenue.plot(kind='bar')

plt.title('Revenue Contribution by Product Category (%)')

plt.bar_label(bars.containers[0], fmt='%.2f%%', fontsize=8)

plt.tight_layout()

plt.show()
```



SQL + PYTHON

Q4. Identify the correlation between product price and the number of times a product has been purchased.

SQL Query

```
SELECT product_id, COUNT(*) AS purchase_frequency,
 ROUND(AVG(price), 2) AS avg_price FROM order_items GROUP BY
 product_id ORDER BY purchase_frequency DESC;
```

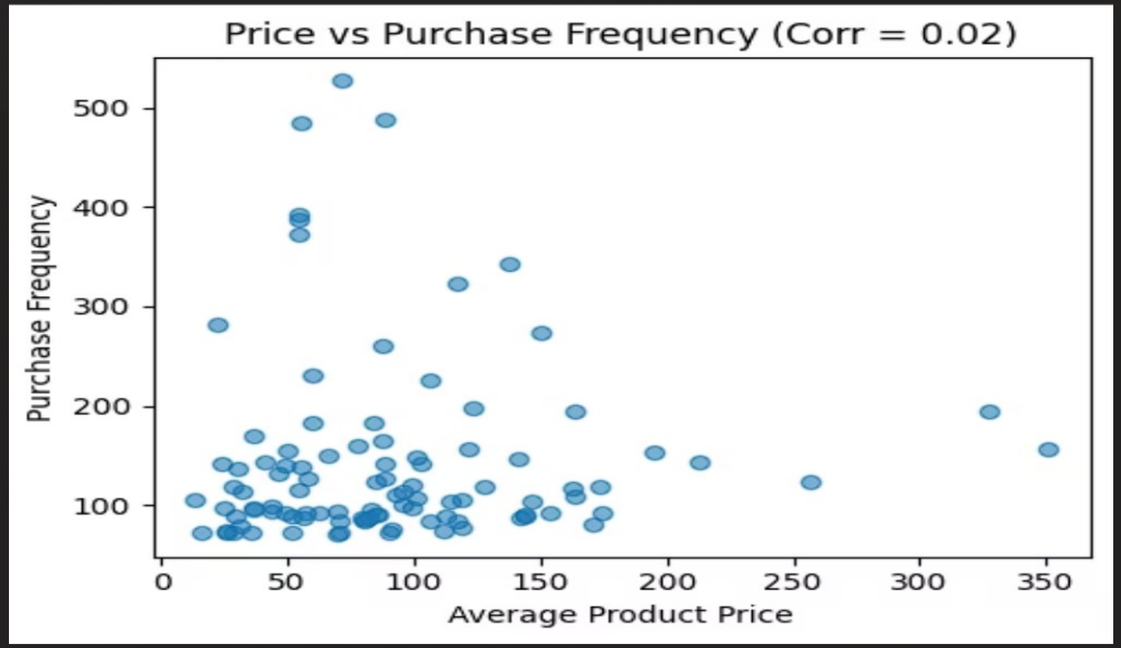
Result Grid | Filter Rows: | Export: | Wrap Cell Content

product_id	purchase_frequency	avg_price
aca2eb7d00ea1a7b8ebd4e68314663af	527	71.36
99a4788cb24856965c36a24e339b6058	488	88.17
422879e10f46682990de24d770e7f83d	484	54.91
389d119b48cf3043d311335e499d9c6b	392	54.7
368c6c730842d78016ad823897a372db	388	54.27
53759a2ecddad2bb87a079a1f1519f73	373	54.66
d1c427060a0f73f6b889a5c7c61f2ac4	343	137.65
53b36df67ebb7c41585e8d54d6772e08	323	116.67

Result 10

Python Query

```
plt.figure(figsize=(5,4))

plt.scatter(
    top_products['avg_price'],
    top_products['purchase_frequency'],
    alpha=0.6)

plt.title(f'Price vs Purchase Frequency (Corr =
{corr_value:.2f})')

plt.xlabel('Average Product Price')

plt.ylabel('Purchase Frequency')
plt.tight_layout()
plt.show()
```



SQL + PYTHON

Q5. Calculate the total revenue generated by each seller, and rank them by revenue.

SQL Query

```
SELECT seller_id, SUM(price) AS total_revenue, RANK() OVER
(ORDER BY SUM(price) DESC) AS seller_rank FROM
order_items GROUP BY seller_id;
```

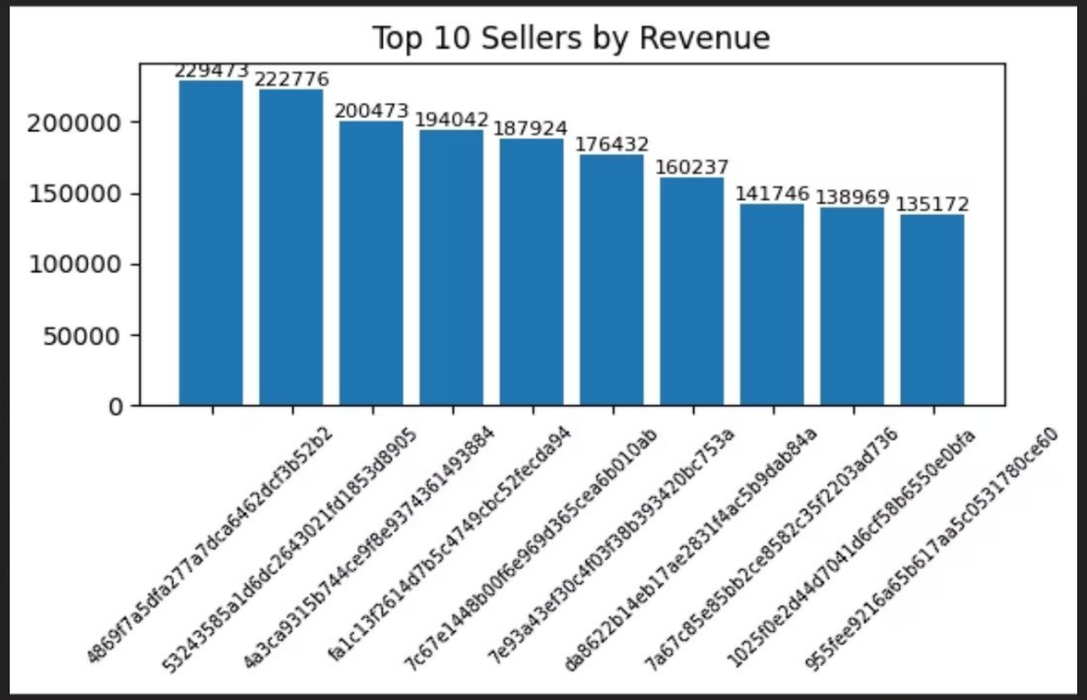
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

seller_id	total_revenue	seller_rank
4869f7a5dfa277a7dca6462dcf3b52b2	229472.6299999981	1
53243585a1d6dc2643021fd1853d8905	222776.04999999952	2
4a3ca9315b744ce9f8e9374361493884	200472.9199999949	3
fa1c13f2614d7b5c4749cbc52fecda94	194042.02999999846	4
7c67e1448b00f6e969d365cea6b010ab	187923.8899999995	5
7e93a43ef30c4f03f38b393420bc753a	176431.86999999982	6
da8622b14eb17ae2831f4ac5b9dab84a	160236.56999999538	7
7a67c85e85bb2ce8582c35f2203ad736	141745.53000000177	8

Result 11

Python Query

```
plt.figure(figsize=(6,4))
bars = plt.bar(top_sellers['seller_id'],
top_sellers['total_revenue'])
plt.title('Top 10 Sellers by Revenue')
plt.xticks(rotation=45, fontsize=7)
plt.bar_label(bars, fmt='%.0f', fontsize=8)
plt.tight_layout()
plt.show()
```

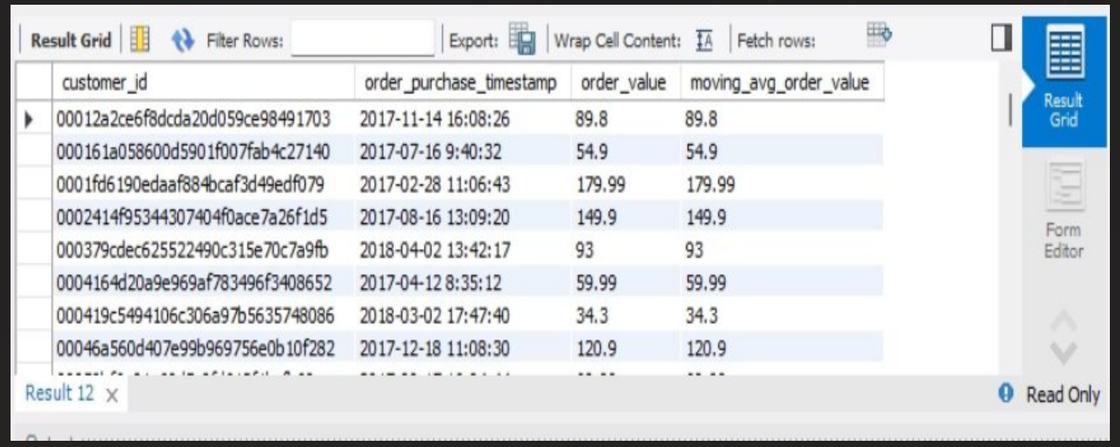


SQL + PYTHON ADVANCED PROBLEM

Q1. Calculate the moving average of order values for each customer over their order history.

SQL Query

```
SELECT o.customer_id, o.order_purchase_timestamp, SUM(oi.price)

 AS order_value, AVG(SUM(oi.price)) OVER ( PARTITION BY o.customer_id
ORDER BY o.order_purchase_timestamp ROWS

 BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW ) AS
moving_avg_order_value FROM orders o JOIN order_items oi ON o.order_id =

 oi.order_id GROUP BY o.customer_id, o.order_id,
o.order_purchase_timestamp ORDER BY o.customer_id,
o.order_purchase_timestamp;
```



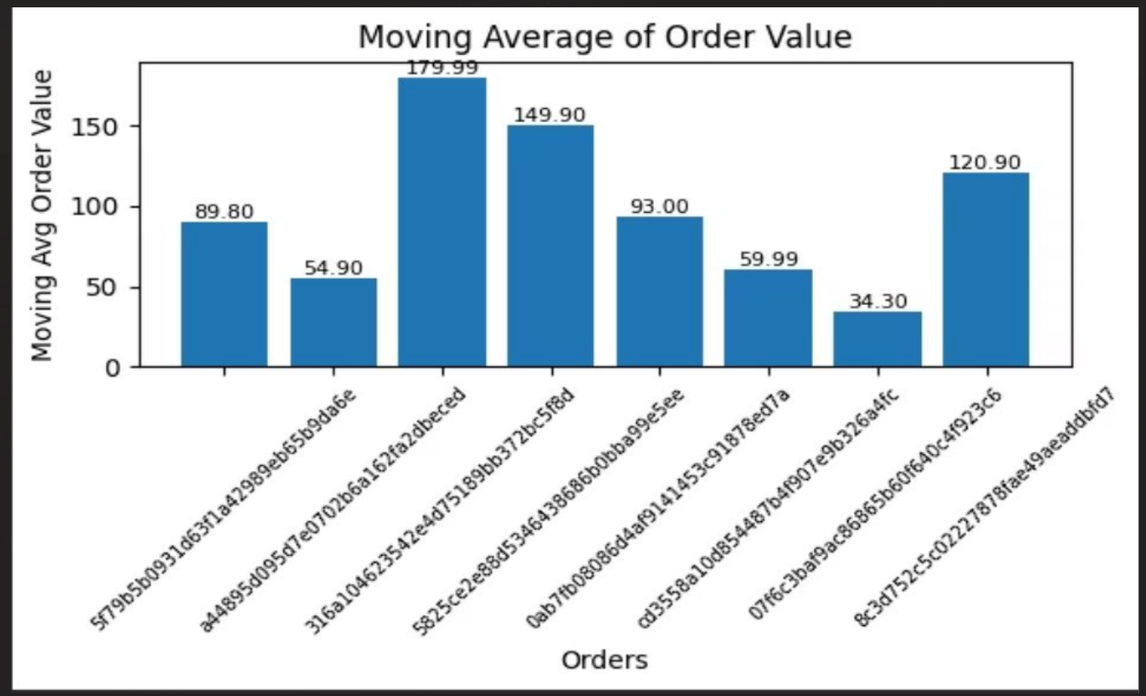
Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

customer_id	order_purchase_timestamp	order_value	moving_avg_order_value
00012a2ce6f8dcda20d059ce98491703	2017-11-14 16:08:26	89.8	89.8
000161a058600d5901f007fab4c27140	2017-07-16 9:40:32	54.9	54.9
0001fd6190edaaf884bcaf3d49edf079	2017-02-28 11:06:43	179.99	179.99
0002414f95344307404f0ace7a26f1d5	2017-08-16 13:09:20	149.9	149.9
000379cdec625522490c315e70c7a9fb	2018-04-02 13:42:17	93	93
0004164d20a9e969af783496f3408652	2017-04-12 8:35:12	59.99	59.99
000419c5494106c306a97b5635748086	2018-03-02 17:47:40	34.3	34.3
00046a560d407e99b969756e0b10f282	2017-12-18 11:08:30	120.9	120.9

Result 12 | Result Grid | Form Editor | Read Only

Python Query

```
plt.figure(figsize=(6,4))
bars =
plt.bar(range(len(sample)),sample['moving_avg_order_value'])plt.ti
tle('Moving Average of Order Value') plt.xlabel('Orders’)

plt.ylabel('Moving Avg Order Value')
plt.xticks(range(len(sample)), sample['order_id'], rotation=45,
fontsize=7)

for bar in bars:
height = bar.get_height()
plt.text(bar.get_x() + bar.get_width()/2,
height,f'{height:.2f}',ha='center',va='bottom',fontsize=8)

plt.tight_layout()
plt.show()
```



SQL + PYTHON

Q2. Calculate the cumulative sales per month for each year.

SQL Query

```
SELECT order_year, order_month, monthly_sales, SUM(monthly_sales) OVER

 ( PARTITION BY order_year ORDER BY order_month ) AS cumulative_sales
FROM ( SELECT YEAR(o.order_purchase_timestamp) AS order_year,

 MONTH(o.order_purchase_timestamp) AS order_month, SUM(oi.price) AS
monthly_sales FROM orders o JOIN order_items oi ON o.order_id =

 oi.order_id GROUP BY YEAR(o.order_purchase_timestamp),
MONTH(o.order_purchase_timestamp) ) t ORDER BY order_year,
order_month;
```

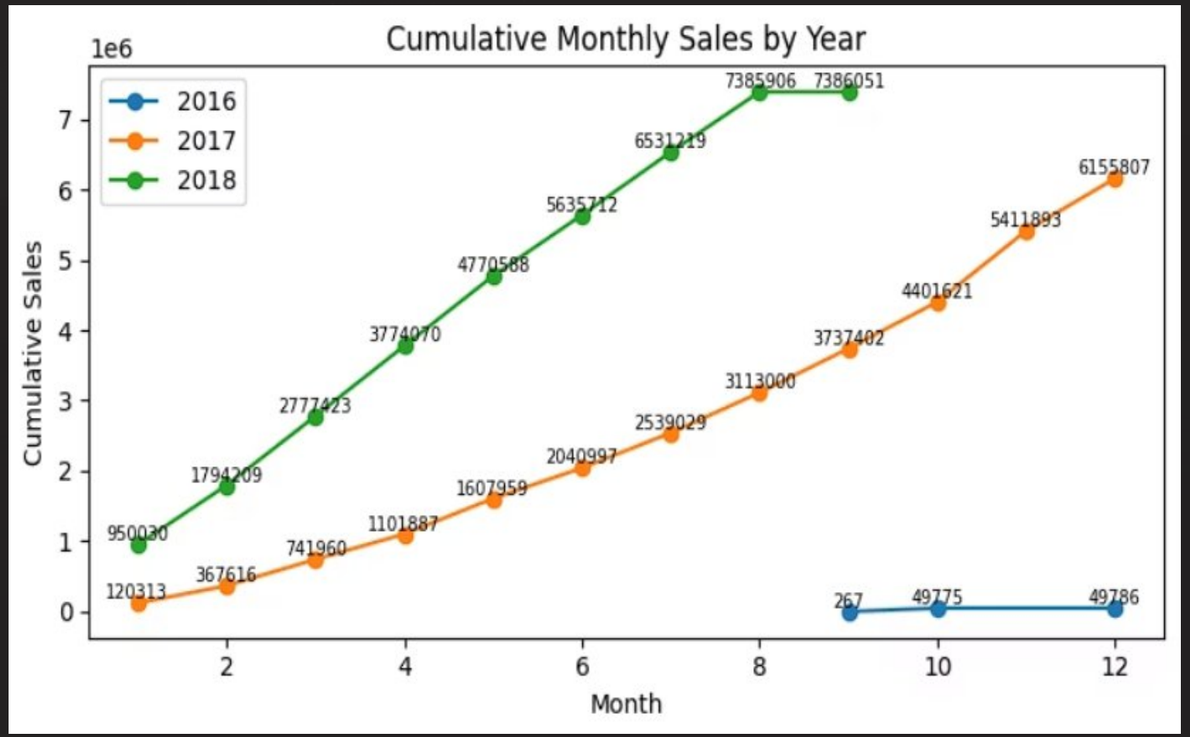
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_year	order_month	monthly_sales	cumulative_sales
2016	9	267.36	267.36
2016	10	49507.66000000018	49775.02000000018
2016	12	10.9	49785.92000000018
2017	1	120312.86999999972	120312.86999999972
2017	2	247303.0199999959	367615.8899999956
2017	3	374344.3000000005	741960.1899999961
2017	4	359927.2299999999	1101887.419999996
2017	5	506071.1400000094	1607958.5600000054

Result 13

Python Query

```
plt.figure(figsize=(7,4))

for y in sorted(monthly_sales['year'].unique()):data =
monthly_sales[monthly_sales['year'] == y]

plt.plot(data['month'],data['cumulative_sales'],
marker='o',label=str(y))

for x, v in zip(data['month'],data['cumulative_sales']):
plt.text(x, v, f'{v:.0f}', fontsize=7, ha='center', va='bottom')
plt.title('Cumulative Monthly Sales by Year')
plt.xlabel('Month')

plt.ylabel('Cumulative Sales')

plt.legend()
plt.tight_layout()
plt.show()
```



SQL + PYTHON

Q3. Calculate the year-over-year growth rate of total sales.

SQL Query

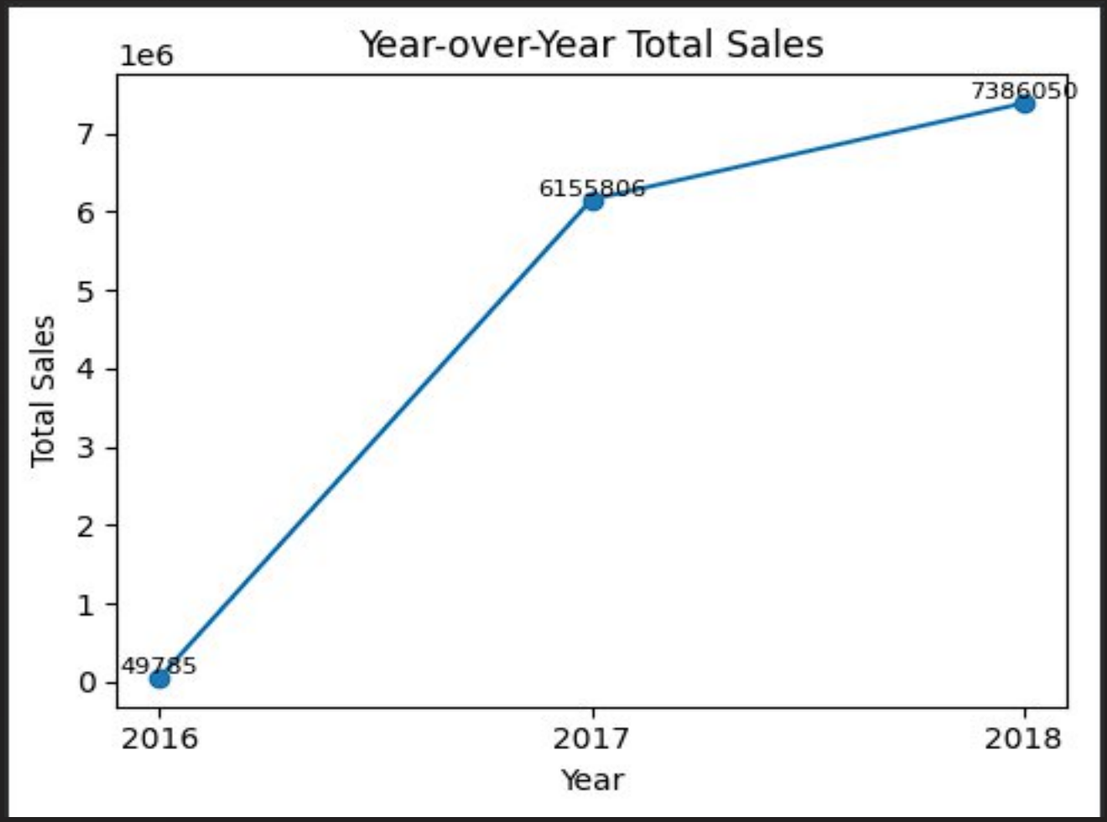
```
SELECT order_year, total_sales, LAG(total_sales) OVER (ORDER BY order_year)
AS previous_year_sales, ROUND( (total_sales - LAG(total_sales)

 OVER (ORDER BY order_year)) * 100.0 / LAG(total_sales) OVER (ORDER BY
order_year), 2 ) AS yoy_growth_percentage

FROM ( SELECT YEAR(o.order_purchase_timestamp) AS order_year,
SUM(oi.price) AS

 total_sales FROM orders o JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY YEAR(o.order_purchase_timestamp) ) yearly_sales ORDER BY
order_year;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_year	total_sales	previous_year_sales	yoy_growth_percentage
2016	49785.92000000019	NULL	NULL
2017	6155806.9800029425	49785.92000000019	12264.55
2018	7386050.800004358	6155806.9800029425	19.99

Python Query

```
plt.figure(figsize=(5,4))
plt.plot(yearly_sales['order_year'],
   yearly_sales['price'],
   marker='o')
for x, y in zip(yearly_sales['order_year'], yearly_sales['price']):
plt.text(x, y, f"{int(y)}", ha='center', va='bottom', fontsize=8)
plt.xticks(yearly_sales['order_year'])
plt.xlabel('Year')
plt.ylabel('Total Sales')
plt.title('Year-over-Year Total Sales')
plt.tight_layout()
plt.show()
yearly_sales
```



SQL + PYTHON

Q4. Calculate the retention rate of customers,defined as the percentage of customer who make another purcahse within 6 month of their purchase.

SQL Query

```
SELECT COUNT(DISTINCT customer_unique_id) AS total_customers,
COUNT(DISTINCT CASE WHEN
 order_purchase_timestamp > first_order_date AND

 order_purchase_timestamp <= DATE_ADD(first_order_date,

 INTERVAL 6 MONTH) THEN customer_unique_id END) AS
retained_customers, ROUND( COUNT(DISTINCT CASE WHEN
order_purchase_timestamp > first_order_date AND

 order_purchase_timestamp <= DATE_ADD(first_order_date, INTERVAL
6 MONTH) THEN customer_unique_id END) * 100.0 / COUNT(DISTINCT
customer_unique_id), 2 ) AS retention_rate_percentage

FROM ( SELECT c.customer_unique_id, o.order_purchase_timestamp,
MIN(o.order_purchase_timestamp) OVER (PARTITION BY
c.customer_unique_id) AS first_order_date FROM orders o JOIN
customers c ON o.customer_id = c.customer_id ) t;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

total_customers	retained_customers	retention_rate_percentage
96096	2234	2.32

Python Query

```
plt.figure(figsize=(4,4))

plt.bar(['Retained Customers'], [retention_rate])

plt.ylabel('Retention Rate (%)')

plt.title('Customer Retention Rate (6 Months)')
plt.text(0, retention_rate, f'{retention_rate}%', ha='center',
va='bottom')

plt.tight_layout()
plt.show()
```

SQL + PYTHON

Q5. Identify the top 3 customers who spent the most money in each year.

SQL Query

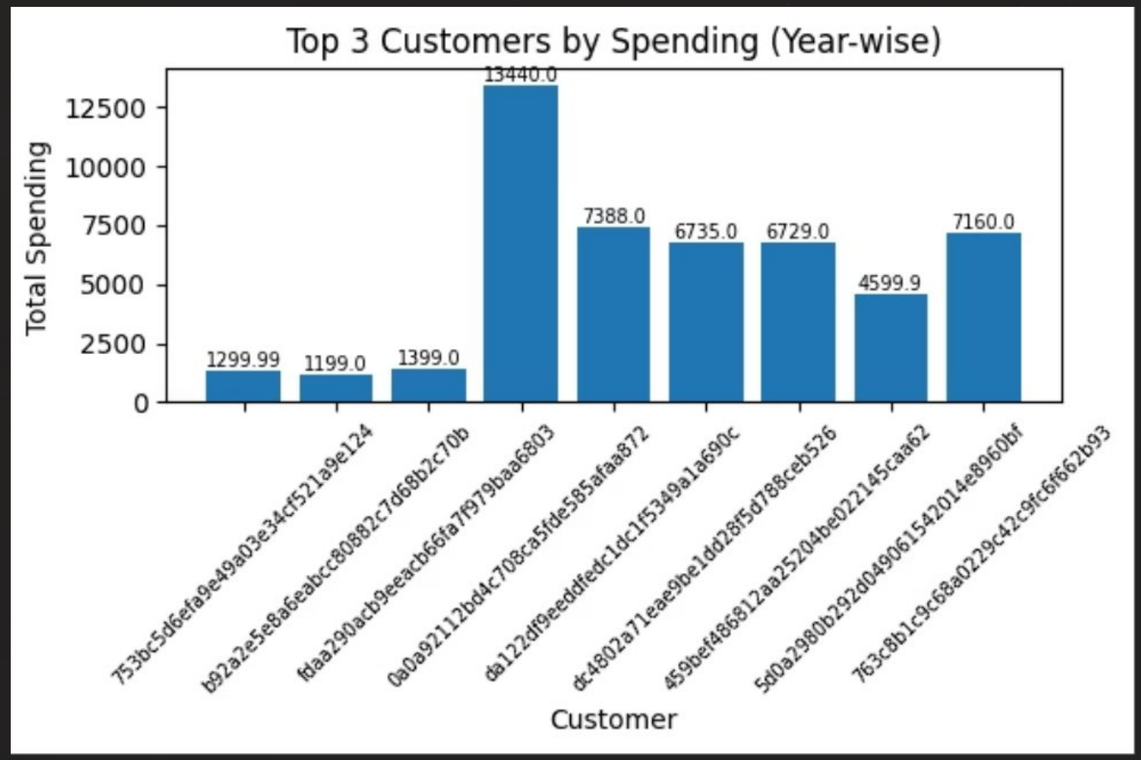
```
SELECT order_year, customer_id, total_spent FROM ( SELECT
YEAR(o.order_purchase_timestamp) AS order_year, o.customer_id,

 SUM(oi.price) AS total_spent, DENSE_RANK() OVER ( PARTITION BY
YEAR(o.order_purchase_timestamp) ORDER BY SUM(oi.price) DESC ) AS

 rank_in_year FROM orders o JOIN order_items oi ON o.order_id =
oi.order_id GROUP BY YEAR(o.order_purchase_timestamp),
o.customer_id ) ranked_customers WHERE rank_in_year <= 3 ORDER BY
order_year, rank_in_year;
```

order_year	customer_id	total_spent
2016	a9dc96b027d1252bbac0a9b72d837fc6	1399
2016	1d34ed25963d5aae4cf3d7f3a4cda173	1299.99
2016	4a06381959b6670756de02e07b83815f	1199
2017	1617b1357756262bfa56ab541c47bc16	13440
2017	c6e2731c5b391845f6800c97401a43a9	6735
2017	3fd6777bbce08a352fddd04e4a7cc8f6	6499
2018	ec5b2ba62e574342386871631fafd3fc	7160
2018	f48d464a0baaea338cb25f816991ab1f	6729

Result 17

Python Query

```
plt.figure(figsize=(6,4))
plt.bar(top3_customers['customer_unique_id'].astype(str),top3_custom
ers['price'])

plt.xlabel('Customer')
plt.ylabel('Total Spending')

plt.title('Top 3 Customers by Spending (Year-wise)')
plt.xticks(rotation=45, fontsize=7)

for i, v in enumerate(top3_customers['price']):
    plt.text(i, v, round(v, 2), ha='center', va='bottom', fontsize=7)

plt.tight_layout()
plt.show()
```



Conclusion: Powering Decisions with Data

Structured Analysis

SQL provided the backbone for structured and accurate data extraction, essential for reliable insights.

Visual Storytelling

Python, through Pandas and Matplotlib, transformed raw data into compelling visualizations, revealing intricate trends.

Enhanced Decision-Making

The combined approach of SQL and Python fortified our decision-making process with validated and clear data.

Real-World Application

This project showcases practical, real-world data analytics skills, preparing for future challenges in e-commerce.

Thank You

Linkedin-https://www.linkedin.com/in/devang-magare-7b266b258/

Github-https://github.com/DevangMagare?tab=repositories

Email- devangmagare@gmail.com